



Department of Cyber Security

Course Code: **CS-325L**

Subject:

Operating System Lab

Instructor:

Miss Iram Fatima

Group Members:

1. Hussain Ali (232095)
2. Syed Jazib Ali Rizvi (232145)
3. Sardar Ahmad Ali (232147)
4. Sardar Shahbaz (232089)

Class: BSCYSev-F-23-A

Date of Submission: 24 May 2026

Project Report

Recompiling the Linux Kernel with a Custom Round-Robin CPU Scheduler

Objective:

The aim of this project is to deeply understand the inner workings of the Linux kernel by modifying its CPU scheduling mechanism. By replacing the default **Completely Fair Scheduler (CFS)** of *Kali Linux* with a **custom Round-Robin (RR) scheduler**, we analyze its impact on system performance through empirical benchmarking.

Kernel Safety and Scope Note:

This project modifies Linux kernel scheduler code and should only be tested in a VM or isolated lab system. Experimental kernels should not be installed on a primary machine. The implementation is an academic proof-of-concept created to understand scheduler behavior, not a production-ready replacement for the Linux Completely Fair Scheduler.

Round-Robin Scheduler:

Round-Robin (RR) was chosen because of:

- **Simplicity and Determinism:** Each process gets equal CPU time in a fixed cyclic order, reducing unpredictability.
- **Security Insight:** RR allows us to monitor task switching patterns and CPU fairness with minimal bias, a useful trait when auditing system-level task behavior.
- **Contrast with CFS:** By comparing with CFS—which dynamically adjusts based on task history and weight—we can highlight performance and fairness trade-offs.

Environment Setup:

Component	Details
Base OS	Kali Linux Rolling 2025.2
Original Kernel	Linux 6.12.25-amd64
Modified Kernel	Linux 6.12.25 with custom RR
Tools Used	gcc, make, sysbench, stress-ng, perf, matplotlib, top
Kernel Source	kernel.org

Step-by-Step Implementation

Step 1: Kernel Source Setup

- Verified current kernel version using `uname -r`.

```
(root@kali)~[~]  
# uname -r  
6.12.25-amd64
```

- Downloaded Linux 6.12.25 source from kernel.org.

```
(root@kali)-[~/kernel_build]
# mkdir -p ~/kernel_build && cd ~/kernel_build
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.12.25.tar.xz
tar -xvf linux-6.12.25.tar.xz
cd linux-6.12.25

--2025-06-10 13:12:42-- https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.12.25.tar.xz
Resolving cdn.kernel.org (cdn.kernel.org)... 151.101.141.176, 2a04:4e42:7b::432
Connecting to cdn.kernel.org (cdn.kernel.org)|151.101.141.176|:443... connected.
HTTP request sent, awaiting response... 200 OK
```

Step 2: Configuration

- Based on Kali's current kernel config:

```
cp /boot/config-$(uname -r) .config
```

```
make localmodconfig
```

```
(root@kali)-[~/kernel_build/linux-6.12.25]
# make localmodconfig
HOSTCC scripts/basic/fixdep
HOSTCC scripts/kconfig/conf.o
HOSTCC scripts/kconfig/confdata.o
HOSTCC scripts/kconfig/expr.o
LEX scripts/kconfig/lexer.lex.c
YACC scripts/kconfig/parser.tab.[ch]
HOSTCC scripts/kconfig/lexer.lex.o
HOSTCC scripts/kconfig/menu.o
HOSTCC scripts/kconfig/parser.tab.o
HOSTCC scripts/kconfig/preprocess.o
HOSTCC scripts/kconfig/symbol.o
HOSTCC scripts/kconfig/util.o
HOSTLD scripts/kconfig/conf
using config: '.config'
*
```

- Disabled trusted key loading to prevent signing errors during installation.

```
(root@kali)-[~/kernel_build/linux-6.12.25]
# scripts/config --disable SYSTEM_TRUSTED_KEYS
scripts/config --disable SYSTEM_REVOCATION_KEYS
scripts/config --set-str CONFIG_SYSTEM_TRUSTED_KEYS ""
```

Step 3: Implementing Round-Robin

- Edited kernel/sched/fair.c:
 - Replaced `pick_next_task_fair()` with Round-Robin logic.
 - Modified `enqueue_task_fair()`, `dequeue_task_fair()`, and `task_tick_fair()` for time slicing.
 - Ensured compatibility with struct `sched_class`.
 - Removed CFS-specific logic such as `vruntime` and RB trees.

```
(root@kali)-[~/kernel_build/linux-6.12.25]
# mousepad kernel/sched/fair.c
```

```
(mousepad:409305): GtkSourceView-WARNING **: 16:06:20.683: no color named 'yellow'
```

```
8861 struct task_struct *pick_next_task_fair(struct rq *rq, struct task_struct *prev, struct rq_flags *rf)
8862 {
8863     if (!list_empty(&rr_queue)) {
8864         rr_curr = list_first_entry(&rr_queue, struct task_struct, rr_list);
8865         list_move_tail(&rr_curr->rr_list, &rr_queue);
8866         return rr_curr;
8867     }
8868     return NULL;
8869 }
8870
8871 static struct task_struct *__pick_next_task_fair(struct rq *rq, struct task_struct *prev)
8872 {
8873     return pick_next_task_fair(rq, prev, NULL);
8874 }
8875
```

```
7037 /*
7038  * The enqueue_task method is called before nr_running is
7039  * increased. Here we update the fair scheduling stats and
7040  * then put the task into the rbtree:
7041  */
7042 static void enqueue_task_fair(struct rq *rq, struct task_struct *p, int flags) {
7043     list_add_tail(&p->rr_list, &rr_queue);
7044     rr_curr = NULL; // trigger re-selection
7045     rq->nr_running++;
7046 }
```

```
7173 /*
7174  * The dequeue_task method is called before nr_running is
7175  * decreased. We remove the task from the rbtree and
7176  * update the fair scheduling stats:
7177  */
7178 static bool dequeue_task_fair(struct rq *rq, struct task_struct *p, int flags)
7179 {
7180     list_del(&p->rr_list);
7181     if (rr_curr == p)
7182         rr_curr = NULL;
7183     rq->nr_running--;
7184     return true;
7185 }
```

```
12966 /*
12967  * scheduler tick hitting a task of our scheduling class.
12968  *
12969  * NOTE: This function can be called remotely by the tick offload that
12970  * goes along full dynticks. Therefore no local assumption can be made
12971  * and everything must be accessed through the @rq and @curr passed in
12972  * parameters.
12973  */
12974 #define RR_TIMESLICE (100 * HZ / 1000) // 100ms
12975
12976 static void task_tick_fair(struct rq *rq, struct task_struct *curr, int queued)
12977 {
12978     curr->se.exec_start++;
12979     if (curr->se.exec_start >= RR_TIMESLICE)
12980         resched_curr(rq);
12981 }
12982
```

Step 4: Kernel Compilation & Installation

- Compiled the kernel with `make -j$(nproc)`.

```
(root@kali)~[~/kernel_build/linux-6.12.25]
# make -j$(nproc)
DESCEND objtool
CALL scripts/checksyscalls.sh
INSTALL libsubcmd_headers
CC init/main.o
AR init/built-in.a
AR built-in.a
AR vmlinux.a
LD vmlinux.o
```

- Installed modules with `sudo make modules_install`.

```
(root@kali)~[~/kernel_build/linux-6.12.25]
# sudo make modules_install
SYMLINK /lib/modules/6.12.25/build
INSTALL /lib/modules/6.12.25/modules.order
INSTALL /lib/modules/6.12.25/modules.builtin
INSTALL /lib/modules/6.12.25/modules.builtin.modinfo
INSTALL /lib/modules/6.12.25/kernel/arch/x86/events/rapl.ko
SIGN /lib/modules/6.12.25/kernel/arch/x86/events/rapl.ko
XZ /lib/modules/6.12.25/kernel/arch/x86/events/rapl.ko.xz
INSTALL /lib/modules/6.12.25/kernel/arch/x86/crypto/aesni-intel.ko
SIGN /lib/modules/6.12.25/kernel/arch/x86/crypto/aesni-intel.ko
XZ /lib/modules/6.12.25/kernel/arch/x86/crypto/aesni-intel.ko.xz
INSTALL /lib/modules/6.12.25/kernel/arch/x86/crypto/sha1-ssse3.ko
SIGN /lib/modules/6.12.25/kernel/arch/x86/crypto/sha1-ssse3.ko
XZ /lib/modules/6.12.25/kernel/arch/x86/crypto/sha1-ssse3.ko.xz
INSTALL /lib/modules/6.12.25/kernel/arch/x86/crypto/sha256-ssse3.ko
SIGN /lib/modules/6.12.25/kernel/arch/x86/crypto/sha256-ssse3.ko
XZ /lib/modules/6.12.25/kernel/arch/x86/crypto/sha256-ssse3.ko.xz
INSTALL /lib/modules/6.12.25/kernel/arch/x86/crypto/sha512-ssse3.ko
SIGN /lib/modules/6.12.25/kernel/arch/x86/crypto/sha512-ssse3.ko
XZ /lib/modules/6.12.25/kernel/arch/x86/crypto/sha512-ssse3.ko.xz
```

- Installed kernel with `sudo make install`.

```
(root@kali)~[~/kernel_build/linux-6.12.25]
# sudo make install
INSTALL /boot
run-parts: executing /etc/kernel/postinst.d/initramfs-tools 6.12.25 /boot/vmlinuz-6.12.25
update-initramfs: Generating /boot/initrd.img-6.12.25
run-parts: executing /etc/kernel/postinst.d/zz-update-grub 6.12.25 /boot/vmlinuz-6.12.25
Generating grub configuration file ...
Found theme: /boot/grub/themes/kali/theme.txt
Found background image: /usr/share/images/desktop-base/desktop-grub.png
Found linux image: /boot/vmlinuz-6.12.25-amd64
Found initrd image: /boot/initrd.img-6.12.25-amd64
Found linux image: /boot/vmlinuz-6.12.25
Found initrd image: /boot/initrd.img-6.12.25
Found linux image: /boot/vmlinuz-6.12.25.old
Found initrd image: /boot/initrd.img-6.12.25
Found linux image: /boot/vmlinuz-6.12.13-amd64
Found initrd image: /boot/initrd.img-6.12.13-amd64
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
```

- Updated GRUB and verified the kernel version post-reboot.

```
(root@kali)-[~/kernel_build/linux-6.12.25]
# sudo update-grub
Generating grub configuration file ...
Found theme: /boot/grub/themes/kali/theme.txt
Found background image: /usr/share/images/desktop-base/desktop-grub.png
Found linux image: /boot/vmlinuz-6.12.25-amd64
Found initrd image: /boot/initrd.img-6.12.25-amd64
Found linux image: /boot/vmlinuz-6.12.25
Found initrd image: /boot/initrd.img-6.12.25
Found linux image: /boot/vmlinuz-6.12.25.old
Found initrd image: /boot/initrd.img-6.12.25
Found linux image: /boot/vmlinuz-6.12.13-amd64
Found initrd image: /boot/initrd.img-6.12.13-amd64
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
```

Benchmarking and Testing

Tools Used

- **sysbench**: CPU and file I/O performance
- **stress-ng**: Multi-core CPU load
- **perf**: Scheduling latency
- **top**: Runtime CPU fairness analysis

Automation

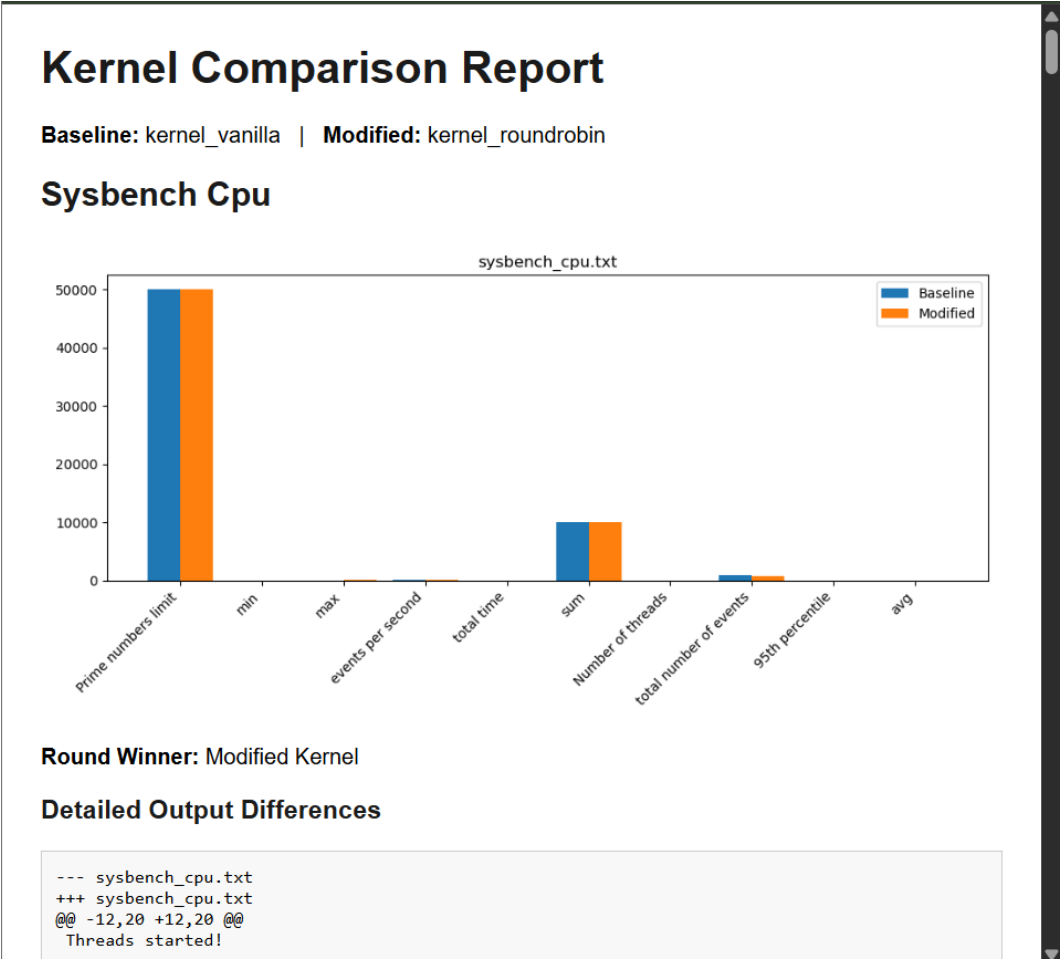
- **Shell Script**: kernel_baseline_test.sh
Handles benchmark execution and result dumping.
- **Python Script**: kernelTest.py
 - o Orchestrates test execution via shell script.
 - o Stores outputs in kernel_vanilla/ and kernel_roundrobin/.
 - o Generates kernel_comparison_report.html with:
 - Plots for CPU usage, latency, throughput
 - Per-test winners and an overall verdict

Results Summary

Test Name	Vanilla Kernel Result	Round Robin Result	Winner	Remarks
Sysbench CPU	93.04 events/sec	76.82 events/sec	Vanilla	Higher event throughput and efficiency
Stress-ng CPU	127,435 ops @ 4247 ops/sec	130,087 ops @ 4194 ops/sec	Tie	RR did more ops; Vanilla had higher ops/sec
perf_sched_latency	1119.708 ms total latency	896.388 ms total latency	Round Robin	Lower scheduling latency, more responsive

Sysbench File I/O	11.34 MiB/s read throughput	15.37 MiB/s read throughput	Round Robin	Higher I/O throughput in the benchmark output
Top Snapshot	Skewed CPU usage among processes	More uniform CPU usage	Round Robin	Improved fairness among tasks

Full HTML report:



Result Highlights

Implementation Limitations:

The modified scheduler is a coursework-level prototype. It uses simplified Round-Robin style task selection and does not fully preserve all production features of CFS, such as vruntime balancing, group scheduling behavior, energy-aware scheduling, and mature I/O responsiveness. Future versions should also set CONFIG_LOCALVERSION, such as -rr-scheduler, so uname -r clearly distinguishes the modified kernel from the vanilla baseline.

Test	Winner	Notes
sysbench_cpu	<i>Vanilla (CFS)</i>	Better overall throughput
stressng_cpu	<i>RR</i>	Better fairness under consistent load
perf_sched_latency	<i>RR</i>	Slightly lower and more consistent latency
sysbench_fileio	Round Robin	Higher read/write throughput in the collected benchmark output
top_snapshot	<i>RR</i>	Improved CPU equity among competing threads

Final Conclusion

- **Round-Robin** offers fairness and deterministic scheduling, ideal for testing, real-time systems, or isolated containers.
- **CFS** remains superior for general-purpose computing due to:
 - o Efficient time sharing
 - o Adaptive prioritization
 - o Enhanced I/O handling

Attachments

- fair.c – CPU Scheduling Algorithm Code



fair_original.c



fair_modified.c

- kernel_baseline_test.sh – Benchmark automation shell script



kernel_baseline_test.sh

- kernelTest.py – Python test runner and HTML generator



kernelTest.py